

Labwork 3 - CUDA

Do Minh Quang - 2540095
University of Science and Technology of Hanoi

1 RGB to Grayscale Conversion

This labwork is about converting an RGB image to Grayscale image on CPU and GPU in order to observe the boost up of GPU over CPU. A 256x256 RGB image is used for the experiment.



Figure 1: RGB image.

The image is first loaded and reshaped into 2D array of shape $(N, 3)$, where N is the total number of pixels.

1.1 CPU Implementation

The CPU implementation processes pixels sequentially in the `for` loop. For each pixel, the average of the RGB channels is calculated and stored in the output array.

```
def rgb2gray_cpu(rgb, gray):  
    for i in range(rgb.shape[0]):  
        g = np.uint8((int(rgb[i, 0]) + int(rgb[i, 1]) +  
                      int(rgb[i, 2])) / 3)  
        gray[i, 0] = gray[i, 1] = gray[i, 2] = g
```

1.2 GPU Implementation

For the GPU version, the flattened RGB image is first transferred from CPU memory to the GPU memory. The CUDA kernel, `rgb2gray_gpu`, is then launched to process pixels in parallel, with each thread handling one pixel. Finally the result grayscale image is copied back to CPU memory for visualization. The initial block size is set to 64 threads.

```
def grayscale_gpu(flatten_rgb, blockSize):
    gridSize = (pixelCount + blockSize - 1) //
        blockSize
    rgb_gpu = cuda.to_device(flatten_rgb)
    gray_gpu = cuda.device_array((pixelCount, 3), np.
        uint8)
    rgb2gray_gpu[gridSize, blockSize](rgb_gpu, gray_gpu
    )
    host_gray_gpu = gray_gpu.copy_to_host()
    return host_gray_gpu
```

2 Results

2.1 CPU and GPU performance

For the quantitative results, GPU boosts the performance by reducing the execution time from 0.1841 seconds on the CPU to 0.0872 seconds on the GPU. This corresponds to a speedup of approximately 2 times.

For the qualitative result, both implementations produced visually consistent grayscale images, as shown in Figure 2.



(a) CPU grayscale output.



(b) GPU grayscale output.

Figure 2: Comparison of grayscale images produced by the CPU and GPU.

2.2 Impact of block size

The GPU process is also evaluated with different block sizes to investigate their impact on execution time. The results are presented in Figure 3.

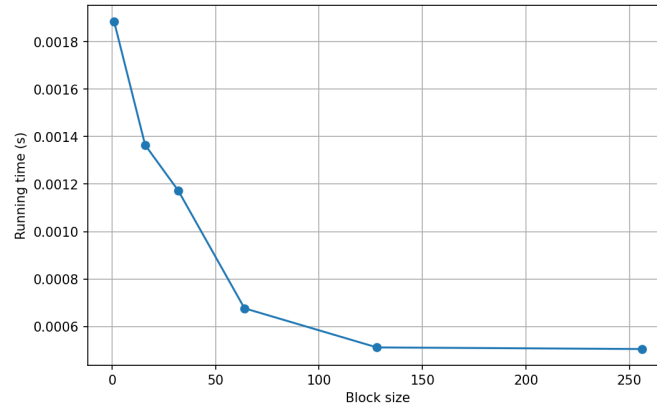


Figure 3: GPU execution time for different block sizes.

The measured running time generally decreases as the block size increases. Smaller blocks require more blocks to process the same number of pixels, potentially increasing scheduling overhead. Larger blocks can improve execution efficiency by reducing this overhead. However, the perfor-

mance gain becomes marginal between block size of 128 and 256. Among the tested configurations, a block size of 256 achieved the shortest execution time of 0.000505 seconds, while a block size of 1 resulted in the longest time of 0.001885 seconds.